Arquitectura DevSecOps para SaaS ERP/POS Multi-Inquilino: Estrategias de Seguridad, Fricción Inteligente y Escalabilidad en.NET 8 y PostgreSQL

El diseño y la implementación de una plataforma de Software as a Service (SaaS) orientada al segmento Business-to-Business (B2B), con capacidades de Planificación de Recursos Empresariales (ERP) y Punto de Venta (POS) multi-inquilino, multi-sucursal y multi-rubro, exige una arquitectura que reconcilie dos fuerzas aparentemente opuestas: la seguridad extrema y la fluidez operativa. En el entorno minorista, la velocidad de procesamiento en el mostrador es el pilar de la viabilidad del negocio; cualquier latencia inducida por protocolos de seguridad obstructivos resulta en una degradación inaceptable de la experiencia de usuario (UX) y, en última instancia, en la pérdida de clientes.[1]

El presente reporte técnico exhaustivo expone un marco de referencia arquitectónico basado en el paradigma de "Fricción Inteligente" y "Defensa en Profundidad". Las medidas de seguridad deben operar de manera asíncrona, progresiva y transparente, garantizando que el ecosistema construido sobre.NET 8 (Web API), PostgreSQL (con esquemas JSONB y Row-Level Security) y React (Progressive Web App) resista vectores de ataque avanzados sin comprometer la eficiencia transaccional.

1. Identidad y Control de Acceso (Fricción Inteligente)

La gestión de identidades en un ecosistema SaaS B2B constituye la primera línea de defensa y, simultáneamente, el mayor punto de fricción potencial. Los procesos de registro tradicionales, caracterizados por formularios extensos y políticas de contraseñas draconianas, elevan drásticamente las tasas de abandono (churn) durante la fase de adquisición.[3] La arquitectura moderna exige una transición hacia mecanismos de identidad que prioricen la conversión sin sacrificar la postura de seguridad.

Onboarding Fluido: Perfilamiento Progresivo y Autenticación Sin Contraseña

El principio de "Fricción Inteligente" postula que las barreras de validación deben introducirse únicamente en proporción directa al riesgo de la acción solicitada. Para la fase de registro (Onboarding), la metodología óptima es el Perfilamiento Progresivo (Progressive Profiling). Esta estrategia consiste en solicitar al usuario no técnico (ej. el dueño de un comercio) exclusivamente la información mínima viable para aprovisionar el inquilino (tenant) y conceder

acceso inicial al sistema.[3]

En lugar de presentar un formulario monolítico que demande la configuración fiscal, la estructura de sucursales y la creación de roles en una sola sesión, el sistema difiere la recolección de estos metadatos para etapas posteriores.[5] A medida que el usuario interactúa con módulos específicos del ERP, la plataforma solicita contextualmente la información pertinente.[7] Este enfoque reduce la carga cognitiva, acelera el tiempo hasta la obtención de valor (Time-to-Value) y construye un nivel de confianza incremental con el cliente.[1]

Para erradicar la fricción asociada a las contraseñas—que constituyen la causa principal de brechas de seguridad por credenciales comprometidas y un centro de costos masivo en tickets de soporte—la arquitectura debe adoptar un enfoque de Autenticación Sin Contraseña (Passwordless Authentication).[4] Utilizando las capacidades de ASP.NET Core 8, se recomienda una estrategia bifurcada:

1. **Acceso Inicial mediante Enlaces Mágicos (Magic Links) u OTP:** Durante el onboarding, el usuario ingresa únicamente su correo electrónico. El backend de.NET 8 genera un token criptográfico de un solo uso, con una ventana de expiración sumamente corta (ej. 10 minutos), y lo transmite asíncronamente a la bandeja de entrada del usuario.[9] Al hacer clic en el enlace, el usuario es autenticado instantáneamente. Esta validación de correo electrónico asíncrona sirve un doble propósito: verifica la propiedad del canal de comunicación (Identity Verification) y elimina la necesidad de establecer y recordar una contraseña inicial.[11]
2. **Adopción de Passkeys (WebAuthn/FIDO2) para el Mostrador:** Una vez que el usuario accede al sistema, se activa un flujo para registrar una Passkey.[13] Las Passkeys, integradas nativamente en.NET 8 mediante bibliotecas como fido2-net-lib o las nuevas extensiones de Identity (.AddPasskeys()), aprovechan el hardware criptográfico del dispositivo del usuario (ej. el chip TPM en Windows Hello o el Secure Enclave en dispositivos móviles) para generar un par de claves asimétricas.[13] La clave pública se almacena en PostgreSQL, mientras que la privada nunca abandona el dispositivo.[13] En futuras interacciones, especialmente críticas en un entorno POS de ritmo rápido, el cajero puede iniciar sesión de forma casi instantánea mediante biometría (huella dactilar o reconocimiento facial) o un PIN local, logrando una autenticación resistente al phishing y libre de fricción.[16]

Protección contra Ataques de Fuerza Bruta y Abuso de API

La eliminación de contraseñas no exime a los endpoints de autenticación (como la solicitud de Magic Links) de ser objetivos de ataques automatizados, incluyendo *Credential Stuffing*, enumeración de usuarios y ataques de Denegación de Servicio (DoS).[4] La protección debe implementarse en la capa de middleware para rechazar el tráfico malicioso antes de que consuma recursos de procesamiento o sature la base de datos.

ASP.NET Core 8 introduce un middleware de limitación de tasa

(Microsoft.AspNetCore.RateLimiting) altamente optimizado y nativo, que reemplaza la necesidad de bibliotecas de terceros.[19] Para proteger la plataforma, se debe implementar una política de limitación estricta utilizando algoritmos específicos según el contexto de la ruta:

Algoritmo de Rate Limiting	Mecanismo	Caso de Uso en ERP/POS
Token Bucket	Mantiene un "cubo" de tokens que se repone a una tasa constante. Permite ráfagas cortas de solicitudes hasta agotar el cubo.	**Endpoints de Autenticación (/login, /magic-link):** Absorbe intentos legítimos rápidos, pero bloquea ataques de fuerza bruta sostenidos.[21]
Fixed Window	Establece un límite estricto de solicitudes dentro de una ventana de tiempo predefinida (ej. 100 reqs/min).	**APIs Públicas y Webhooks:** Controla el consumo general del ancho de banda y protege contra el raspado de datos (scraping).[20]
Concurrency Limiter	Limita el número de solicitudes que se ejecutan simultáneamente, sin importar la ventana de tiempo.	**Operaciones Intensivas (Reportes complejos, importaciones masivas):** Previene el agotamiento de los hilos del servidor de aplicaciones (Thread Pool Starvation).[18]

Para un entorno B2B multi-inquilino, la limitación de tasa debe ser inteligente y particionada. Utilizando PartitionedRateLimiter, el sistema puede aplicar restricciones basadas en la dirección IP para usuarios no autenticados, y basadas en el tenant_id para usuarios autenticados.[23] Esto garantiza que si una sucursal experimenta un mal funcionamiento (ej. un escáner de código de barras enviando miles de peticiones erróneas), se aplique un bloqueo temporal (código de estado HTTP 429 *Too Many Requests*) exclusivamente a ese inquilino o IP, aislando el impacto y asegurando el principio de "uso justo" (Fair Usage) para el resto de la plataforma.[18] En arquitecturas escaladas horizontalmente, este middleware debe acoplarse con un almacenamiento distribuido como Redis para sincronizar los contadores entre múltiples instancias del servidor.[20]

Adicionalmente, para proteger intentos de acceso maliciosos contra cuentas específicas,

ASP.NET Core Identity proporciona opciones nativas de bloqueo temporal (LockoutOptions). Se debe configurar el sistema para bloquear una cuenta durante un período incremental (ej. 5 minutos, luego 15, luego 60) tras un número reducido de intentos fallidos (ej. 5 intentos), disuadiendo eficazmente el *Credential Stuffing*.[25]

Gestión de Tokens (JWT): Arquitectura PWA, Cookies vs LocalStorage

El manejo seguro de la sesión en una Progressive Web App construida con React representa uno de los debates arquitectónicos más críticos. En un entorno POS, el cajero no puede permitirse que la sesión expire sorpresivamente mientras procesa el pago de un cliente. La elección del mecanismo de almacenamiento para el JSON Web Token (JWT) determina la vulnerabilidad del sistema ante ataques de Cross-Site Scripting (XSS) y Cross-Site Request Forgery (CSRF).

El almacenamiento del JWT de acceso directamente en el LocalStorage o SessionStorage del navegador es una práctica categorizada como de alto riesgo.[26] El ecosistema de React depende extensamente de paquetes NPM de terceros. Si un atacante logra comprometer una dependencia e inyectar código JavaScript malicioso (ataque a la cadena de suministro), dicho script tendrá acceso irrestricto al LocalStorage mediante la API del DOM (window.localStorage.getItem(...)).[26] Esto permite la exfiltración silenciosa del JWT hacia un servidor externo, otorgando al atacante la capacidad de suplantar la identidad del usuario sin desencadenar ninguna alerta de seguridad.[27]

Por otro lado, almacenar el JWT en cookies HTTP tradicionales mitiga el riesgo de XSS si se configuran adecuadamente, pero históricamente exponía la aplicación a ataques CSRF, donde un sitio malicioso fuerza al navegador del usuario a emitir una petición falsificada hacia el API, arrastrando automáticamente las cookies de sesión.[28]

El consenso de seguridad contemporáneo ("Gold Standard") para aplicaciones de página única (SPA) y PWAs es el **Patrón de Tokens Divididos (Split Token Architecture) con Renovación Silenciosa (Silent Refresh)** [27]:

1. **Access Token en Memoria Efímera:** El servidor de.NET 8 emite un JWT de acceso con un ciclo de vida sumamente corto (ej. 5 a 15 minutos). La aplicación React recibe este token en el cuerpo (payload) de la respuesta JSON y lo almacena estrictamente en la memoria volátil de la aplicación (estado de React, Redux, Zustand o Context API).[27] Al no persistir en el disco del navegador, el token es invisible para ataques XSS pasivos; si el usuario cierra o recarga la pestaña, el token en memoria se destruye.[27]
2. **Refresh Token en Cookie HttpOnly:** Simultáneamente, el backend emite un token de actualización (Refresh Token) opaco y de larga duración (ej. 7 a 30 días). Este token no se entrega en el cuerpo JSON, sino a través de un encabezado Set-Cookie fuertemente fortificado con las directivas HttpOnly, Secure y SameSite=Strict.[27] La directiva HttpOnly vuelve a la cookie criptográficamente invisible para cualquier script de JavaScript en el

cliente, neutralizando por completo el vector de ataque XSS.[32] La directiva SameSite=Strict instruye al motor del navegador para que nunca adjunte esta cookie en solicitudes iniciadas desde dominios de terceros, mitigando eficazmente los ataques CSRF.[34]

3. **Flujo de Renovación Silenciosa mediante Interceptores de Axios:** Para garantizar la continuidad operativa en el POS sin requerir reautenticaciones constantes, se implementa una lógica de intercepción en el cliente HTTP (ej. Axios). Cuando el Access Token expira, el API de.NET rechaza la siguiente solicitud con un código de estado 401 Unauthorized. El interceptor de Axios captura este error *antes* de que llegue al componente de la interfaz de usuario.[32] En este punto, el interceptor suspende y encola la solicitud original, y automáticamente emite una nueva solicitud asíncrona hacia el endpoint de renovación (/api/auth/refresh).[32] Dado que la PWA y el API operan en el mismo dominio, el navegador adjunta instintivamente la cookie HttpOnly que contiene el Refresh Token.[32] El servidor valida el Refresh Token, evalúa su asociación con la sesión activa en la base de datos (permitiendo la revocación remota de sesiones) y, si es válido, responde con un nuevo Access Token efímero.[32] El interceptor actualiza el token en la memoria de React, reescribe el encabezado Authorization: Bearer de las solicitudes encoladas y las retransmite.[32] Este ciclo completo ocurre en milisegundos, siendo totalmente imperceptible para el cajero. Si el Refresh Token resulta inválido o ha expirado, el interceptor intercepta el fallo consecuente, purga el estado de la aplicación y redirige al usuario a la interfaz de autenticación de manera limpia y controlada.[32]

Esta triangulación arquitectónica asegura una robustez extrema contra las vulnerabilidades de la web moderna, preservando al mismo tiempo la ininterrupción absoluta de los flujos de trabajo críticos en el punto de venta.

2. Seguridad Multi-Inquilino y Base de Datos (PostgreSQL)

En una arquitectura de Software as a Service, la segregación hermética de la información entre diferentes empresas, franquicias y sucursales es el pilar sobre el cual se sostiene la confianza del cliente y el cumplimiento normativo. Una brecha en la cual un inquilino logra acceder a los datos financieros, métricas de inventario o bases de datos de clientes de otro inquilino—fenómeno conocido como "Data Bleed" o vulnerabilidad BOLA (Broken Object Level Authorization)—representa una falla sistémica catastrófica.[39]

Capas Defensivas más allá del Row-Level Security (RLS)

El paradigma del esquema compartido y la base de datos compartida (Shared Database, Shared Schema) es la arquitectura predeterminada para el despliegue ágil de sistemas multi-inquilino debido a su extrema eficiencia de costos y facilidad de escalabilidad.[40] En este contexto, la implementación del Row-Level Security (RLS) de PostgreSQL es una salvaguarda fundacional. RLS permite definir políticas a nivel del motor de base de datos que actúan como

predicados invisibles anexados a cada consulta SQL. Mediante la configuración de políticas (ej. CREATE POLICY tenant_isolation ON orders USING (tenant_id = current_setting('app.current_tenant_id'));), el motor garantiza que, sin importar las deficiencias en la lógica de la aplicación.NET, un inquilino jamás podrá consultar ni modificar registros pertenecientes a otro.[42]

Sin embargo, depositar la integridad absoluta de la arquitectura en una única capa de RLS introduce riesgos sistémicos sutiles, particularmente en arquitecturas de alto rendimiento. El punto de falla más crítico reside en la **contaminación del pool de conexiones (Connection Pool Contamination)**.[39] Las aplicaciones.NET 8 (a través de Npgsql) o los balanceadores externos como PgBouncer mantienen un conjunto de conexiones físicas abiertas hacia PostgreSQL para evitar la penalización de latencia inherente al proceso de establecimiento de conexión TCP/TLS. Para que RLS funcione, la aplicación típicamente emite un comando SET app.current_tenant_id = 'uuid'; al adquirir una conexión del pool.[39] Si la aplicación sufre una excepción no controlada, un tiempo de espera (timeout), o simplemente omite restablecer el contexto antes de devolver la conexión al pool, la siguiente solicitud HTTP que reutilice esa conexión física específica heredará el contexto de seguridad del inquilino anterior.[39] Esta falla silenciosa puede provocar que el sistema del inquilino B comience a facturar con la identidad y los datos del inquilino A.

Para construir un ecosistema impenetrable, la seguridad multi-inquilino debe estratificarse en capas concurrentes:

1. **Aislamiento de Sesión Transaccional Estricto:** Nunca se deben utilizar comandos de estado de sesión globales y persistentes. La inyección del contexto de inquilino debe ejecutarse de forma encapsulada y temporal. En PostgreSQL, esto se logra forzando que todas las consultas operen dentro de una transacción explícita (BEGIN... COMMIT) y utilizando el comando SET LOCAL.[39] A diferencia de SET, la variable definida por SET LOCAL es volátil; el motor de base de datos la descarta automáticamente en el momento exacto en que la transacción finaliza (ya sea por un COMMIT o un ROLLBACK). Esto garantiza criptográficamente que la conexión retorne al pool en un estado neutro y prístino, inmunizando al sistema contra la contaminación cruzada.[39] Utilizando los Interceptores de Comandos de Entity Framework Core en.NET 8, este patrón se puede inyectar de manera automatizada antes de cada ejecución, eliminando la dependencia en la memoria de los desarrolladores.
2. **Aislamiento en Caché (Keyspace Collisions):** En arquitecturas de alto rendimiento, gran parte de la carga de lectura se desvía a sistemas de caché distribuidos como Redis.[39] Una vulnerabilidad común es el solapamiento del espacio de claves. Si la aplicación consulta el caché con una clave genérica como GET product_123, corre el riesgo de devolver el inventario de un inquilino diferente.[39] La arquitectura debe imponer un prefijado forzoso y determinista en la capa de abstracción del caché (ej. GET tenant_A:product_123), asegurando una segmentación virtual de los datos volátiles que espeje la protección de la base de datos subyacente.[39]

3. **Aislamiento de Esquema Híbrido para Entornos Empresariales:** A medida que la plataforma SaaS escala hacia el segmento empresarial (Mid-Market y Enterprise), las exigencias de cumplimiento normativo (Compliance) se intensifican. Ciertos inquilinos corporativos exigirán aislamiento físico de datos para asegurar el aislamiento de las copias de seguridad, encriptación independiente y evasión del fenómeno de "vecino ruidoso" (donde la carga intensiva de un inquilino afecta el rendimiento de otro).[42] La arquitectura debe soportar un modelo híbrido "Bridge": los inquilinos pequeños y medianos residen en un esquema compartido protegido por RLS, mientras que los inquilinos empresariales se aprovisionan en esquemas independientes de PostgreSQL (Schema-per-Tenant) dentro de la misma instancia.[40] La lógica de conexión enrutadora en.NET 8, apoyándose en la resolución de dependencias, selecciona dinámicamente la cadena de conexión o el esquema en función de la jerarquía de la suscripción del inquilino autenticado.

Protección JSONB: Prevención de Inyecciones NoSQL y Desbordamientos

La arquitectura del sistema ERP multi-rubro exige una flexibilidad en la estructura de datos que los modelos relacionales rígidos no pueden proveer. La capacidad de PostgreSQL para almacenar e indexar eficientemente documentos JSON estructurados mediante el tipo de dato binario JSONB es el núcleo de esta flexibilidad.[48] Permite que el sistema gestione atributos dinámicos dispares en una misma tabla maestra (por ejemplo, una tabla Products donde un registro tiene metadatos de "talla y color" para prendas de vestir, y otro tiene "voltaje y amperaje" para ferretería) sin requerir costosas modificaciones de esquema (ALTER TABLE) o complejas tablas de modelo Entidad-Atributo-Valor (EAV).[49]

No obstante, esta flexibilidad introduce riesgos sustanciales inherentes a las bases de datos NoSQL y documentales. Cuando la aplicación acepta cargas útiles (payloads) JSON dinámicas y arbitrarias desde el frontend de React para insertarlas en un campo JSONB, abre vectores de ataque como la Inyección NoSQL (NoSQLi) y ataques de desbordamiento de recursos (Denial of Service).[51]

Aunque las consultas parametrizadas suministradas por Npgsql y Entity Framework Core aíslan la entrada del usuario de la sintaxis SQL estructural, previniendo inyecciones clásicas (SQLi) como la concatenación maliciosa de sentencias, **no previenen** la manipulación de la lógica de evaluación profunda en campos JSON.[51] Un atacante malicioso, entendiendo que el motor de búsqueda utiliza operadores JSON, podría enviar una carga útil estructurada que incluya operadores lógicos anidados (como equivalentes JSON a $ne, $gt, o inyecciones de expresiones regulares complejas) diseñados para evadir la lógica de autenticación o para provocar una evaluación exponencial que bloquee el hilo de procesamiento de la CPU de PostgreSQL (ReDoS - Regular Expression Denial of Service).[51]

La estrategia de mitigación en C#.NET 8 requiere un enfoque multicapa, combinando

restricciones físicas, validación estructural y sanitización lógica:

1. **Contención en el Borde (Middleware Kestrel):** El primer perímetro de defensa es prevenir ataques volumétricos. Atacantes o scripts defectuosos podrían intentar enviar objetos JSON de varios gigabytes con niveles masivos de anidamiento para agotar la memoria del servidor (Out of Memory) y el almacenamiento. La configuración del servidor web Kestrel en.NET 8 debe establecer umbrales estrictos limitando el tamaño máximo del cuerpo de la solicitud (MaxRequestBodySize) y la profundidad máxima de lectura permitida en el formateador JSON (JsonSerializerOptions.MaxDepth).[54]
2. **Validación Dinámica con JSON Schema:** Dado que el contenido exacto del campo JSONB varía dependiendo del contexto del rubro comercial, la validación estática en C# mediante el enlace a clases fuertemente tipadas (POCOs) es insuficiente o inviable.[55] La solución robusta es externalizar las reglas de integridad a documentos **JSON Schema**.[57] La base de datos mantendrá una tabla auxiliar que almacena la definición oficial de validación (el JSON Schema) correspondiente a la categoría del producto o al inquilino. Cuando el controlador de la API recibe la actualización del producto, el flujo es el siguiente:
   - El motor recupera el JSON Schema autorizado.
   - El payload JSON entrante se analiza como un JsonNode (System.Text.Json).[57]
   - Utilizando una biblioteca de alto rendimiento (como JsonSchema.Net o NJsonSchema), el sistema evalúa estructuralmente el JsonNode contra el esquema.[57]
   - El esquema debe estar configurado de forma estricta (ej. "additionalProperties": false) para rechazar cualquier propiedad no definida, evitando inyecciones de parámetros inesperados.[59] Validará la longitud de cadenas de texto, formatos numéricos y la obligatoriedad de los atributos críticos. Si el documento viola cualquiera de las reglas, la solicitud es abortada devolviendo un código 400 Bad Request, impidiendo que datos anómalos o malformados se persistan en el campo JSONB de PostgreSQL.[57]
3. **Sanitización Estricta contra XSS:** Incluso si el JSON cumple perfectamente con la estructura y las longitudes esperadas, los valores de las cadenas de texto (ej. la "descripción" de un ítem) podrían contener fragmentos de código inyectados, como etiquetas <script>. Si estos valores se almacenan ciegamente en el JSONB y luego se envían de vuelta a la aplicación React de otro usuario, se podría desencadenar un ataque Cross-Site Scripting persistente (Stored XSS).[54] Como última capa antes de la persistencia, el backend debe iterar sobre los valores textuales del objeto y procesarlos a través de herramientas especializadas (como la biblioteca HtmlSanitizer de.NET) que identifiquen y neutralicen de forma segura los marcadores HTML o scripts incrustados, garantizando la esterilidad de los metadatos.[54]

3. Seguridad a Nivel de API (.NET 8) y Frontend (React)

La resiliencia de la plataforma SaaS ante la vasta gama de vulnerabilidades documentadas en el estándar OWASP Top 10 depende de la simbiosis defensiva entre las directivas del backend de.NET 8 y la configuración del entorno de ejecución del cliente en el navegador. Las barreras

de contención deben orquestarse cuidadosamente en el archivo de inicio de la aplicación y extenderse hacia la arquitectura de red y la interfaz de usuario.

Configuraciones Obligatorias en Program.cs y Contramedidas OWASP

El motor de ASP.NET Core 8 proporciona un arsenal de herramientas nativas para robustecer la comunicación HTTP. La aplicación debe configurarse para inyectar sistemáticamente Encabezados de Seguridad (Security Headers) en cada respuesta, instruyendo al navegador del cliente sobre los límites permisibles de operación y previniendo ataques de inyección, manipulación de contenido y secuestro.[60] Para centralizar y garantizar la exactitud de estas configuraciones en Program.cs, se recomienda encarecidamente la implementación de middlewares especializados como NetEscapades.AspNetCore.SecurityHeaders u OwaspHeaders.Core.[62]

El perfil de seguridad obligatorio abarca:

1. **HSTS (HTTP Strict Transport Security):** La aplicación debe aplicar la política Strict-Transport-Security con un valor de duración extenso (ej. max-age=31536000; includeSubDomains).[61] Esta directiva exige al navegador que todas las comunicaciones futuras con el dominio se realicen exclusivamente a través de canales cifrados (HTTPS). Esta configuración es vital para anular intentos de degradación de protocolo y secuestro de red por intermediarios locales (Man-in-the-Middle).[61]
2. **Protección contra Clickjacking:** El secuestro de clics ocurre cuando un atacante envuelve fraudulentamente la PWA dentro de un marco transparente (<iframe> u <object>) en una página web maliciosa, engañando al usuario (como a un cajero distraído) para que interactúe con controles invisibles. Para erradicar este vector, la API debe despachar el encabezado X-Frame-Options: DENY (o SAMEORIGIN si se requiere incrustación interna legítima), complementado inexcusablemente por la directriz moderna de CSP frame-ancestors 'none'.[60]
3. **Protección de Contenido (MIME Sniffing):** El encabezado X-Content-Type-Options: nosniff debe figurar de manera inquebrantable.[60] Esto impide que navegadores antiguos infieran y ejecuten contenido malicioso (como scripts JavaScript camuflados con extensiones de archivos de imagen) que haya sido introducido por atacantes eludiendo los filtros de subida de archivos.[60]
4. **CORS (Cross-Origin Resource Sharing):** La configuración en.NET 8 (builder.Services.AddCors()) debe rechazar el uso del comodín universal (*). Las políticas deben ser absolutamente restrictivas (AllowSpecificOrigins), enlistando explícitamente los dominios exactos desde los cuales se sirven las PWAs de las sucursales, y definiendo de forma granular los métodos permitidos (GET, POST, PUT) y encabezados autorizados. Adicionalmente, la configuración debe autorizar el intercambio de credenciales (AllowCredentials) de manera estricta para viabilizar el transporte de la cookie HttpOnly del Refresh Token.[67]
5. **Política de Seguridad de Contenido (CSP - Content Security Policy):** Constituye el

elemento defensivo supremo de la PWA React contra ataques de inyección (XSS) y extracción de datos.[61] Una política restrictiva (Content-Security-Policy) declara los orígenes fidedignos desde los cuales la aplicación puede cargar recursos (scripts, hojas de estilo, imágenes y datos asíncronos). Deberá definirse como default-src 'self' para confinar la carga a su propio origen, y proscribir rotundamente el uso de JavaScript incrustado directamente en el HTML o rutinas dinámicas peligrosas como eval() mediante la supresión terminante del atributo 'unsafe-inline' en la configuración de script-src.[60] Esta disposición estructural aborta de facto cualquier esfuerzo de un atacante por inducir la ejecución de secuencias de comandos no autorizadas en el ecosistema React, sirviendo de respaldo definitivo incluso si la validación del backend falla.[60]

Protección del Sistema POS: Neutralización de Manipulación de Estado en el Cliente

El ecosistema de comercio electrónico y aplicaciones de punto de venta (POS) fundamentadas en tecnologías web confronta un vector de ataque generalizado y crítico: la manipulación asimétrica de los parámetros y la lógica de negocio en el entorno del cliente. Dado que la totalidad del código JavaScript, la memoria volátil de React y el Document Object Model (DOM) de la PWA residen y se ejecutan en el dispositivo físico, un actor malicioso o un cajero con propósitos fraudulentos detenta un control incontrovertible sobre dicho entorno.[70] Mediante el despliegue de extensiones de navegador invasivas o la manipulación directa a través de las Herramientas para Desarrolladores (DevTools), es trivial interceptar las solicitudes en tránsito o alterar el estado local de los componentes.[70] Un escenario representativo implica la manipulación en tiempo real del objeto del carrito de compras, adulterando el precio unitario de un artículo de alto valor a una fracción de centavo, forzando un descuento ilícito o modificando la cantidad de productos exentos de impuestos antes de despachar el comando de facturación hacia el servidor.[72]

Para inmunizar la plataforma frente a este espectro de amenazas, la arquitectura del sistema debe anclarse en la premisa innegociable de la "Confianza Cero" (Zero Trust) sobre la integridad del cliente. Ninguna rutina de validación o cálculo matemático ejecutado en el ecosistema React (JavaScript) ostenta autoridad funcional; las validaciones del lado del cliente poseen un propósito única y exclusivamente enfocado en la experiencia de usuario (UX), proveyendo retroalimentación inmediata, validando la completitud sintáctica de los campos o mitigando viajes de red fútiles, pero carecen de validez de seguridad.[27]

La robustez arquitectónica contra la manipulación de estado exige los siguientes patrones en el diseño del API.NET 8:

1. **Autoridad Absoluta del Servidor en la Transacción:** El diseño del endpoint de facturación o registro de pedidos jamás debe aceptar o procesar variables financieras enviadas como parte del cuerpo (payload) de la petición desde la aplicación React.[73] El cliente POS tiene permitido transmitir únicamente una estructura minimalista y

desprovista de metadatos de costo, conformada por los identificadores absolutos e inmutables de los productos (ProductID) y las cantidades solicitadas.[75] Al interceptar esta matriz, la lógica del backend.NET 8 ignora sistemáticamente cualquier intento externo de definir el precio y consulta la base de datos de PostgreSQL en tiempo real para recabar la fuente de la verdad (Source of Truth) referente a precios unitarios, reglas impositivas vigentes y esquemas de descuento aplicables a la sucursal.[73] La orquestación y el recalculo de la aritmética tributaria y los totales agregados ocurren en el entorno aislado y confiable del servidor, neutralizando cualquier estratagema de manipulación subyacente.

2. **Mitigación de Inconsistencia por Latencia Física (Patrón Idempotente):** Las operaciones de misión crítica en un mostrador POS de flujo intenso están inexorablemente expuestas a inestabilidades del hardware físico de red (microcortes de conexión Wi-Fi, latencias intermitentes en la capa móvil). Si un cajero activa el proceso de pago, experimenta un tiempo de espera (timeout) aparente y presiona repetidamente el comando de cobro, la aplicación podría emitir múltiples peticiones superpuestas. En un escenario ingenuo, el backend interpretaría estas señales como solicitudes transaccionales discretas, ejecutando descuentos paralelos en los niveles de inventario y produciendo una doble o triple facturación ruinosa.[76] Para extirpar esta vulnerabilidad operativa y mantener la coherencia financiera, el diseño del Web API instaura la obligatoriedad estructural del patrón de **Claves de Idempotencia** (Idempotency Keys).[76]
   - **Generación Temprana:** Al momento en que el usuario ensambla el carrito en la PWA, la aplicación React forja un identificador de unicidad universal (UUID v4) y lo anexa de forma persistente como un encabezado HTTP personalizado (ej. Idempotency-Key: <UUID>) en la solicitud POST remitida al servidor.[76]
   - **Gestión Centralizada y Concurrente:** El ecosistema.NET intercepta la solicitud y verifica inmediatamente la clave en un almacén de datos distribuido y de altísima velocidad, idealmente Redis.[80] Si el UUID no figura en el repositorio, la aplicación procede a procesar la transacción íntegra (deducciones de stock, validaciones, confirmación a la base de datos PostgreSQL) y subsiguientemente, inscribe un registro en caché vinculando el UUID con el código de estado de respuesta original (ej. HTTP 201 Created) y la anatomía del payload resultante, dotándolo de un ciclo de vida o *Time-to-Live* (TTL) prudente (ej. 24 horas).[77]
   - **Rechazo Silencioso:** Si, debido a reintentos ansiosos en el mostrador o bucles generados por inestabilidad de red, el servidor recibe peticiones subsecuentes adjuntando la misma clave, el mecanismo detecta la duplicación, puentea por completo la capa de ejecución de la lógica empresarial (protegiendo el inventario) y devuelve graciosamente el resultado histórico encriptado en caché, asegurando a la PWA y al cajero que la transacción primaria prosperó.[77] Este diseño asegura fiabilidad operativa ante la volatilidad física del entorno minorista e inmuniza al sistema contra ataques deliberados de repetición.

4. Auditoría y Trazabilidad (Escalable)

En infraestructuras de nivel empresarial como un SaaS ERP B2B, mantener una traza de auditoría integral, omnipresente y forense no constituye un mero agregado analítico, sino un imperativo sistémico crítico.[81] La trazabilidad exige responder de manera inalterable y precisa a interrogantes vitales: "Qué cajero autorizó un descuento inusual", "Quién purgó registros de inventario crítico" y "Bajo qué contexto temporal e IP se alteraron las configuraciones fiscales". No obstante, la implementación ingenua del registro histórico de acciones CRUD choca frontalmente con la necesidad de preservar el alto rendimiento en el mostrador del POS. La generación síncrona de bitácoras detalladas que capturen y escriban los "estados previos y posteriores" (before/after deltas) en la base de datos principal, en conjunción con cada solicitud originada por miles de terminales concurrentes, desata una rápida congestión de I/O, multiplicando los tiempos de respuesta transaccionales a niveles intolerables.[82]

Trazabilidad de Alto Rendimiento sin Latencia: Patrón Job Offloading y Channels

Para resolver este dilema arquitectónico y orquestar un esquema de auditoría masivamente escalable y no bloqueante en el ecosistema.NET 8, se debe implementar el patrón de diseño de asincronía delegada conocido como *Job Offloading*, respaldado por estructuras de concurrencia de memoria ultrarrápidas.[84] Históricamente, este escenario se gestionaba mediante la serialización y encolamiento manual hacia sistemas de mensajería externos (como RabbitMQ) o en colas de memoria simples (ConcurrentQueue<T>). Sin embargo, para aplicaciones que operan en un mismo proceso, el estándar técnico supremo en.NET radica en el aprovechamiento de la biblioteca nativa System.Threading.Channels.[83]

El diseño opera según una arquitectura sofisticada de productor-consumidor [85]:

1. **Captura y Encolamiento (Productor):** Durante la rutina transaccional primaria del Controlador Web API, bibliotecas robustas e integradas al ciclo de vida del ORM, como Audit.NET.PostgreSql adosada a Entity Framework Core, recopilan automáticamente el linaje del evento: extraen la identidad del inquilino autenticado (vía JWT), la firma del endpoint HTTP, el sello de tiempo (timestamp) y los deltas JSON (modificaciones precisas en las entidades subyacentes).[86] Crucialmente, en lugar de forzar a la aplicación a emitir un costoso comando INSERT hacia PostgreSQL como parte de la solicitud sincrónica del cliente, el módulo transfiere asíncronamente el objeto de telemetría inyectándolo ("produciendo") en un Canal de memoria.[83]
2. **Mitigación de Riesgos por Sobreflujo (Backpressure):** La estructura del canal no debe configurarse como ilimitada, ya que un pico intempestivo de operaciones asíncronas (como la sincronización masiva de inventarios a altas horas de la noche) podría desbordar la capacidad del recolector de basura (Garbage Collector) y detonar un colapso del proceso (Out of Memory Exception).[83] En cambio, se instituye un canal delimitado (Channel.CreateBounded<AuditEvent>). Esta configuración orquesta de forma autónoma una política de "contrapresión" (Backpressure); si la canalización llega al límite configurado de eventos no resueltos, puede indicarle a los hilos de producción que pausen

graciosamente su actividad de inserción (mediante directrices como BoundedChannelFullMode.Wait), asegurando la estabilidad y homeostasis del servidor web Kestrel bajo circunstancias de extrema tensión.[83]

3. **Procesamiento Asíncrono Continuo (Consumidor):** Desvinculado del ciclo de vida crítico y temporal de las peticiones HTTP (las cuales se liberan inmediatamente, retornando latencias sub-milisegundo hacia el frontend React de la sucursal), un servicio permanente incorporado nativamente en ASP.NET Core (IHostedService o BackgroundService) asume el rol de consumidor en segundo plano.[83] Este proceso residente supervisa el conducto e iterativamente drena y extrae lotes completos de las métricas de registro auditadas mediante transmisiones asíncronas no bloqueantes (await foreach con lecturas conjuntas a través de ReadAllAsync()).[83] Al concentrar las entradas en lotes controlados, el servicio acomete operaciones de inserción masiva y altamente optimizadas (Bulk Inserts) en la base de datos PostgreSQL, amortiguando dramáticamente el impacto en la red e interactuando fluida y coordinadamente con el soporte de cierres garantizados del framework (Graceful Shutdown), lo cual afianza el volcado seguro de transacciones postreras ante la suspensión o reinicio del aplicativo.[83]

Almacenamiento Masivo: Particionamiento Declarativo en PostgreSQL

A pesar de optimizar el vector de transferencia, el desafío perenne del registro de auditoría recae inevitablemente en el almacenamiento. Las tablas orientadas a trazabilidad son estructuras que exhiben una fenomenología de inserción aditiva continua (append-only), generando millones de hileras a un compás vertiginoso.[82] Una tabla estructural monolítica pronto se transforma en un cuello de botella fatal: el abultamiento progresivo de sus índices erosiona velozmente las tasas de asimilación de los INSERT masivos enviados por el servicio de.NET, y los ineludibles procesos sistémicos de la base de datos, como los barridos del recolector de desechos lógicos (VACUUM y autovacuum), devienen en eventos excesivamente dilatados e interruptivos que gravan el rendimiento integral de la instancia compartida por todos los inquilinos.[82]

Para desmantelar este estrangulamiento, se debe eludir las arquitecturas clásicas de archivo fragmentado e instrumentalizar íntegramente la poderosa funcionalidad de **Particionamiento Declarativo (Declarative Partitioning)** que proporciona de forma madura PostgreSQL.[82] La estrategia requiere que la estructura de la base se despliegue a través del delineado inicial de una gran tabla central de referencias ("Parent Table") y el dictado de normativas explícitas para segmentar su materialidad física a partir de una técnica de segregación basada en rango temporal (PARTITION BY RANGE (created_at) o fecha de marca).[81]

Este modelo se instaura habitualmente acoplado con herramientas de automatización o rutinas cronometradas que engendran segmentos periódicos proactivamente (e.g., audit_events_2026_03, audit_events_2026_04).[81] Durante la descarga de flujos que efectúa el servicio perimetral de.NET, el gestor de asignaciones inteligente de PostgreSQL orientará los eventos de cada sucursal irremisiblemente hacia el sub-bloque que le concierna con extrema

presteza, confinando la densidad de la transacción.[82]

La instrumentación transita hacia tres prodigios operacionales directos en la escalabilidad y subsistencia del sistema:

1. **Exclusión de Particiones a Nivel Optimizado (Partition Pruning):** Cuando el personal administrativo efectúe indagaciones analíticas forenses desde las interfaces maestras del ERP (por ejemplo, demandando la inspección exhaustiva de fluctuaciones de precios o supresiones de facturas para un día puntual de una semana concreta), el planeador nativo de búsquedas del optimizador de consultas de PostgreSQL suprimirá en una exhalación el análisis superfluo sobre las docenas de bloques particionados subyacentes carentes del contexto temporal adscrito a la petición (Pruning). Esta omisión garantiza la reducción del examen al sub-receptáculo objetivo, posibilitando mermas astronómicas en el tiempo de búsqueda y agilidad transicional inalterable para tablas abismalmente extensas.[82]
2. **Mitigación Quirúrgica en las Rutinas de Mantenimiento:** Cada bloque mensual o cíclico forjado opera manteniendo su constelación de índices segmentados. Como derivación natural y ventajosa, el peso inherente a las secuencias de reconstrucción y las exigentes sesiones automatizadas del servicio autovacuum se reducen dramáticamente y se circunscriben individualmente, operando con eficiencia superior.[82]
3. **Depuración Inmediata en el Ciclo de Vida del Dato:** La gestión histórica obligatoria para satisfacer los protocolos de auditoría se transforma radicalmente. El paradigma antiguo conminaba a la imposición de procesos nocturnos enviando extensas sentencias DELETE FROM altamente disolventes y generadoras de inflación o hinchamiento lógico de los bloques del disco subyacente (Bloat).[82] En contraste, dentro de la arquitectura particionada, un script perimetral automatizado acata la normativa de retención corporativa desprendiendo o cercenando velozmente la partición caduca mediante un simple edicto DDL (Data Definition Language) de manipulación abstracta, como la caída (DROP TABLE audit_events_2024_03).[82] Esta maniobra orquesta una ablación masiva de gigabytes en meros milisegundos, erradicando para siempre los fragmentos difuntos sin someter las cabeceras lectoras del sistema de producción a una asfixia de ciclos ni menoscabar a otros inquilinos concurrentes de la infraestructura SaaS ERP/POS.[82]

Entregable: Plan de Acción y Herramientas

Para instrumentar efectivamente esta arquitectura, se propone el siguiente mapa de integración, segmentado en fases de madurez y referenciando los paquetes nativos o mantenidos por la comunidad recomendados para el ecosistema propuesto.

Área Tecnológica	Fase Básica / Obligatoria	Fase Avanzada / Escalable

Identidad (Onboarding y Auth)	**ASP.NET Core Identity (APIs nativas):** Utilizar MapIdentityApi para emisión de JWT estándar y gestión de sesiones. Implementar Onboarding guiado por correo con flujos de **Magic Links** (OTP de un solo uso). Middleware UseRateLimiter (Fixed Window) global.[18]	**Passkeys y WebAuthn:** Integrar flujos biométricos robustos (Windows Hello, FaceID) para cajeros utilizando fido2-net-lib o la extensión nativa AddPasskeys(). Rate Limiting (Token Bucket) particionado por IP e inquilino utilizando caché distribuido Redis para mitigar ataques multi-nodo.[13]
Gestión de la Sesión en React PWA	**Almacenamiento Aislado:** Patrón de tokens divididos. El Access Token corto habita estrictamente en estado transitorio (Context API / Zustand). El Refresh Token persiste oculto bajo un encapsulado férreo en una **Cookie configurada con las insignias HttpOnly, Secure, y SameSite=Strict** para demoler asaltos mixtos XSS/CSRF.[27]	**Renovación Autónoma Interceptada:** Empleo de una red de Interceptores en el cliente de comunicaciones HTTP (Axios Interceptors). La rutina capta la insidia asíncrona de las excepciones 401, forjando simultáneamente colas controladas y resolviendo el *Silent Refresh* ininterrumpidamente, preservando así la pureza de flujos transaccionales al nivel de la superficie de cobro.[32]
Seguridad de la Base de Datos y Aislamiento Multi-Tenant	**Filtrado Forzoso Nivel Motor:** Configuración fundacional activando el dictamen imperativo Row-Level Security (RLS) con la formulación unánime de CREATE POLICY en la estructura tabular base, orillando toda sentencia SQL para que sea	**Aislamiento Robusto Transaccional:** Control exhaustivo y saneamiento en el engranaje del *Connection Pooling* (PgBouncer) estableciendo las identidades efímeras inyectadas rigurosamente mediante el empleo circunscripto de la directiva

	restringida indefectiblemente por tenant_id.[42]	SET LOCAL intramuros en operaciones del DbContext, conjurando letalmente cualquier filtración o corrupciones entrelazadas. Para corporaciones Enterprise (cumplimiento extremo), bifurcación en una arquitectura híbrida Bridge Schema-per-Tenant.[39]
Estabilización e Integridad de Metadatos (JSONB)	**Supresión Volumétrica Temprana:** Establecimiento del dique nativo de control de flujo activando reguladores en las arterias del Middleware Kestrel delimitando MaxRequestBodySize. Mapeo restrictivo con uso irrestricto de clases DTO antes de someter cargas inciertas.[54]	**Esquematización Dinámica (JSON Schema Validation):** Para frustrar asaltos *NoSQL Injections*, delegación hacia librerías validadoras profundas, ejemplificada por la adopción de JsonSchema.Net para auscultar coercitivamente las variables heterogéneas ("additionalProperties": false) de cada estirpe sectorial. Neutralización residual del veneno incrustado invocando procesadores pasivos como el HtmlSanitizer de C# previo al anclaje en PostgreSQL.[54]
API Defensiva y Protección de Operatoria POS	**Contramuros de Respuesta Web:** Acorazamiento universal instaurando la adopción temprana del compendio bibliográfico NetEscapades.AspNetCore.SecurityHeaders en Program.cs.	**Soberanía y Fiabilidad Server-Side:** Inhabilitación en el procesamiento orgánico sobre mutaciones efectuadas en el cliente, centralizando la verdad matemática en PostgreSQL. Mitigación contra redundancias

	Engendramiento categórico de los edictos HSTS, las fronteras *CORS*, la oclusión a inferencias falsas (*nosniff*), y el encubrimiento de clics (*DENY*).[60]	transaccionales dictada por inestabilidades ambientales de red desplegando los candados distributivos y reincidentes de Idempotency-Keys (UUIDs), refrendados instantáneamente mediante la conjunción con un baluarte intermedio y de control asíncrono como Redis. Promulgación taxativa de una Política CSP purgada del letal 'unsafe-inline'.[61]
Arquitectura de Auditoría Forense y Desempeño Escalonado	**Estructura del Registro:** Despliegue primario del registrador universal instrumentado bajo el control automático de la entidad Audit.NET.PostgreSql. Secuenciación rigurosa de las manipulaciones en cascadas de bases temporales para captación de asimetrías subyacentes post-manipulación de información de facturación.[86]	**Patrón Job Offloading con Canales y Particionado Declarativo:** Mutación de las cadencias entrelazadas apelando a System.Threading.Channels (Channel.CreateBounded). Desviación operativa hacia componentes satelitales que operan ajenos a los carriles críticos del aplicativo, apalancándose mediante consumidores BackgroundService. Estratificación de la masiva información resultante en el motor PostgreSQL empleando el cisma programático del Particionamiento Declarativo PARTITION BY RANGE, orquestando y aniquilando al unísono los lastres estructurales en métricas *Pruning* y purgas indoloradas históricas en su

		recolección o remoción mediante truncamientos (DROP PARTITION).[82]

Este compendio arquitectónico edifica una respuesta estructurada capaz de cimentar una robustez imbatible ante el abanico sistémico de embates inherentes al universo web actual. Asegura intrínsecamente la viabilidad y agilidad sin concesiones, posibilitando a cada cliente la ejecución fluida, resguardada e inviolable que la vertiginosidad de su línea de cajas de cobro exige cada segundo de su jornada de desempeño.

Fuentes citadas

1. Best Practices for Self-Service Customer Onboarding in SaaS - EverAfter.ai, acceso: marzo 12, 2026, https://www.everafter.ai/blog/best-practices-for-self-service-onboarding
2. Best SaaS Onboarding Software Roundup 2025 - GTM Works, acceso: marzo 12, 2026, https://www.saasboost.io/blog/best-saas-onboarding-software-roundup-2023
3. Progressive Profiling 101: Right User Info at the Right Time - Descope, acceso: marzo 12, 2026, https://www.descope.com/learn/post/progressive-profiling
4. Building Secure Authentication Faster: When SaaS Teams Should Go Passwordless, acceso: marzo 12, 2026, https://mojoauth.com/blog/building-secure-passwordless-authentication-saas
5. 10 Customer Onboarding Best Practices for 2025 - Mindstamp, acceso: marzo 12, 2026, https://mindstamp.com/blog/customer-onboarding-best-practices
6. 10 Actionable SaaS Onboarding Best Practices for 2025 - Brand.dev, acceso: marzo 12, 2026, https://www.brand.dev/blog/saas-onboarding-best-practices
7. Understand How Progressive Profiling Works - Auth0 Docs, acceso: marzo 12, 2026, https://auth0.com/docs/manage-users/user-accounts/user-profiles/progressive-profiling
8. SaaS Authentication Best Practices in 2026 - supastarter, acceso: marzo 12, 2026, https://supastarter.dev/blog/saas-authentication-best-practices
9. Why B2B SaaS needs passwordless authentication | Scalekit Blog, acceso: marzo 12, 2026, https://www.scalekit.com/blog/passwordless-authentication-b2b-saas
10. Magic Links Tutorial Secure Passwordless Login Made Simple - SuperTokens, acceso: marzo 12, 2026, https://supertokens.com/blog/magiclinks
11. Passwordless Authentication Onboarding Done Right - hoop.dev, acceso: marzo 12, 2026, https://hoop.dev/blog/passwordless-authentication-onboarding-done-right/
12. Implementing Passwordless Authentication with ASP.NET Core Identity | ABP.IO, acceso: marzo 12, 2026, https://abp.io/community/articles/implementing-passwordless-authentication-with-asp.net-core-identity-c25l8koj
13. How to Implement Passkey in ASP.NET Core with Fido2-net-lib? | Syncfusion

Blogs, acceso: marzo 12, 2026, https://www.syncfusion.com/blogs/post/passkey-in-asp-dotnet-core-with-fido2
14. Go Passwordless with Passkeys | Curity Identity Server, acceso: marzo 12, 2026, https://curity.io/product/authentication/passwordless-authentication/passkeys/
15. Passkeys in ASP.NET Core - C# Corner, acceso: marzo 12, 2026, https://www.c-sharpcorner.com/article/passkeys-in-asp-net-core/
16. Going Passwordless - A Practical Guide to Passkeys in ASP.NET Core - Maarten Balliauw, acceso: marzo 12, 2026, https://www.youtube.com/watch?v=-BV2BCWQGiQ
17. Going Passwordless - A Practical Guide to Passkeys in ASP.NET Core - YouTube, acceso: marzo 12, 2026, https://www.youtube.com/watch?v=U4_KcjJOxOE
18. Rate limiting middleware in ASP.NET Core | Microsoft Learn, acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit?view=aspnetcore-10.0
19. How to Implement Rate Limiting in ASP.NET Core 8? - C# Corner, acceso: marzo 12, 2026, https://www.c-sharpcorner.com/article/how-to-implement-rate-limiting-in-asp-net-core-8/
20. How to Implement Rate Limiting in ASP.NET Core 8, acceso: marzo 12, 2026, https://oneuptime.com/blog/post/2026-01-25-aspnet-core-rate-limiting/view
21. Rate Limiting in ASP.NET Core 8 — A Practical Developer's Guide - Yaseer Arafat, acceso: marzo 12, 2026, https://blog.yaseerarafat.com/bulletproof-your-net-apis-with-native-rate-limiting-4271f12e7707
22. Exploring Rate Limiting Middleware in .NET 8 - DEV Community, acceso: marzo 12, 2026, https://dev.to/sai_ram_413f7c92188cc2220/exploring-rate-limiting-middleware-in-net-8-3c81
23. Using ASP.NET Core Rate Limiter Middleware - ServiceStack, acceso: marzo 12, 2026, https://servicestack.net/posts/asp-rate-limiter-middleware
24. Rate limiting middleware samples - Microsoft Learn, acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit-samples?view=aspnetcore-10.0
25. Secure ASP.NET Core against credential stuffing - Stack Overflow, acceso: marzo 12, 2026, https://stackoverflow.com/questions/54744885/secure-asp-net-core-against-credential-stuffing
26. Is it safe to store a JWT in localStorage with ReactJS? - Stack Overflow, acceso: marzo 12, 2026, https://stackoverflow.com/questions/44133536/is-it-safe-to-store-a-jwt-in-localstorage-with-reactjs
27. JWT Storage in React: Local Storage vs Cookies Security Battle - Cyber Sierra, acceso: marzo 12, 2026, https://cybersierra.co/blog/react-jwt-storage-guide/
28. Local Storage Versus Cookies: Which to Use to Securely Store Session Tokens,

acceso: marzo 12, 2026, https://www.pivotpointsecurity.com/local-storage-versus-cookies-which-to-use-to-securely-store-session-tokens/
29. Secure JWT Storage: Best Practices | Syncfusion Blogs, acceso: marzo 12, 2026, https://www.syncfusion.com/blogs/post/secure-jwt-storage-best-practices
30. React - localStorage vs. Cookies: A Love Triangle with Your JWT - DEV Community, acceso: marzo 12, 2026, https://dev.to/igornosatov_15/react-localstorage-vs-cookies-a-love-triangle-with-your-jwt-57o0
31. where better to store jwt ? : r/react - Reddit, acceso: marzo 12, 2026, https://www.reddit.com/r/react/comments/1mr8kav/where_better_to_store_jwt/
32. Creating a JWT Authentication System with HTTP-only Refresh Token Using React and Node.js | by Alper Kilickaya | Medium, acceso: marzo 12, 2026, https://medium.com/@alperkilickaya/creating-a-jwt-authentication-system-with-http-only-refresh-token-using-react-and-node-js-6865f04087ce
33. Storing JWT in local storage : r/cybersecurity - Reddit, acceso: marzo 12, 2026, https://www.reddit.com/r/cybersecurity/comments/1iod2i9/storing_jwt_in_local_storage/
34. JWT in Cookies vs LocalStorage — Which Is Safer? | by Hasan Chinthaka - Medium, acceso: marzo 12, 2026, https://medium.com/@chinthakahasan/jwt-in-cookies-vs-localstorage-which-is-safer-e74602144f3c
35. NextJS SSR - JWT (Access/Refresh Token) Authentication with external Backend, acceso: marzo 12, 2026, https://www.thewidlarzgroup.com/blog/nextjs-ssr---jwt-access-refresh-token-authentication-with-external-backend
36. Token Refresh with Axios Interceptors for a Seamless Authentication Experience - Medium, acceso: marzo 12, 2026, https://medium.com/@velja/token-refresh-with-axios-interceptors-for-a-seamless-authentication-experience-854b06064bde
37. Refresh & Access tokens, SPAs.. cookies.. safe to use httponly cookie for both? - Reddit, acceso: marzo 12, 2026, https://www.reddit.com/r/webdev/comments/fg1zyb/refresh_access_tokens_spas_cookies_safe_to_use/
38. Best way set up protected Route in react using JWT? - Stack Overflow, acceso: marzo 12, 2026, https://stackoverflow.com/questions/79902719/best-way-set-up-protected-route-in-react-using-jwt
39. Multi-Tenant Leakage: When “Row-Level Security” Fails in SaaS | by InstaTunnel - Medium, acceso: marzo 12, 2026, https://medium.com/@instatunnel/multi-tenant-leakage-when-row-level-security-fails-in-saas-da25f40c788c
40. Multi-tenant data isolation with PostgreSQL Row Level Security | AWS Database Blog, acceso: marzo 12, 2026, https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgr

esql-row-level-security/
41. How to Architect a Multi-Tenant SaaS with PostgreSQL RLS — A Deep Dive - Substack, acceso: marzo 12, 2026, https://substack.com/home/post/p-161156695?utm_campaign=post&utm_medium=web
42. Mastering PostgreSQL Row-Level Security (RLS) for Rock-Solid Multi-Tenancy, acceso: marzo 12, 2026, https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/
43. How to Secure Multi-Tenant Data with Row-Level Security in PostgreSQL - OneUptime, acceso: marzo 12, 2026, https://oneuptime.com/blog/post/2026-01-25-row-level-security-postgresql/view
44. Data Isolation in Multi-Tenant Software as a Service (SaaS) - Redis, acceso: marzo 12, 2026, https://redis.io/blog/data-isolation-multi-tenant-saas/
45. SaaS Tenant Isolation: Database, Schema, and Row-Level Security Strategies - KodekX, acceso: marzo 12, 2026, https://kodekx-solutions.medium.com/saas-tenant-isolation-database-schema-and-row-level-security-strategies-7337d2159066?source=rss------architecture-5
46. Choose the right PostgreSQL data access pattern for your SaaS application - AWS, acceso: marzo 12, 2026, https://aws.amazon.com/blogs/database/choose-the-right-postgresql-data-access-pattern-for-your-saas-application/
47. Underrated Postgres: Build Multi-Tenancy with Row-Level Security : r/PostgreSQL - Reddit, acceso: marzo 12, 2026, https://www.reddit.com/r/PostgreSQL/comments/1nk4c3a/underrated_postgres_build_multitenancy_with/
48. Documentation: 18: 8.14. JSON Types - PostgreSQL, acceso: marzo 12, 2026, https://www.postgresql.org/docs/current/datatype-json.html
49. PostgreSQL JSONB in .NET - Marek Sirkovský - Medium, acceso: marzo 12, 2026, https://mareks-082.medium.com/postgresql-jsonb-in-net-25fbcc7b64b2
50. Scaling PostgreSQL for Multi-Tenant SaaS: A Practical and Beginner-Friendly Guide Using Citus | by Mayur Parve | Medium, acceso: marzo 12, 2026, https://medium.com/@parvemayur/scaling-postgresql-for-multi-tenant-saas-a-practical-and-beginner-friendly-guide-using-citus-ec89fbf3c177
51. NoSQL Injection : Understanding the Vulnerability and Remediation | by Ajay Monga | T3CH, acceso: marzo 12, 2026, https://medium.com/h7w/nosql-injection-understanding-the-vulnerability-and-remediation-ea1efcbda26f
52. NoSQL Injection Explained: What It Is and How to Prevent It - Bright Security, acceso: marzo 12, 2026, https://brightsec.com/blog/nosql-injection-explained-what-it-is-and-how-to-prevent-it/
53. How to prevent SQL injection in JSON data interface? - Tencent Cloud, acceso: marzo 12, 2026, https://www.tencentcloud.com/techpedia/127996
54. Protecting Against JSON Injection and Malformed Payloads in ASP.NET Core - C#

Corner, acceso: marzo 12, 2026, https://www.c-sharpcorner.com/article/protecting-against-json-injection-and-malformed-payloads-in-asp-net-core/
55. Mapping JSON | Npgsql Documentation, acceso: marzo 12, 2026, https://www.npgsql.org/doc/types/json.html
56. Issue Configuring `EnableDynamicJson()` on Data Source with a Multi-Tenant API Leveraging Postgres Row-Level Security · Issue #3239 · npgsql/efcore.pg - GitHub, acceso: marzo 12, 2026, https://github.com/npgsql/efcore.pg/issues/3239
57. Json schema validation in .NET - Meziantou's blog, acceso: marzo 12, 2026, https://www.meziantou.net/json-schema-validation-in-dotnet.htm
58. Json Schema Patterns in .NET - Data Object Validation | endjin, acceso: marzo 12, 2026, https://endjin.com/blog/2024/05/json-schema-patterns-dotnet-data-object-validation
59. How to C#: Validating a JSON to a JSON Schema - DEV Community, acceso: marzo 12, 2026, https://dev.to/roelvs/how-to-c-validating-a-json-to-a-json-schema-3mle
60. HTTP Headers - OWASP Cheat Sheet Series, acceso: marzo 12, 2026, https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html
61. OWASP Secure Headers Project, acceso: marzo 12, 2026, https://owasp.org/www-project-secure-headers/
62. Implement security headers for an ASP.NET Core API supporting OpenAPI Swagger UI, acceso: marzo 12, 2026, https://damienbod.com/2024/10/21/implement-security-headers-for-an-api-supporting-openapi-swagger-ui/
63. Home | OwaspHeaders.Core - GitHub Pages, acceso: marzo 12, 2026, https://gaprogman.github.io/OwaspHeaders.Core/
64. OwaspHeaders.Core 10.2.0 - NuGet, acceso: marzo 12, 2026, https://www.nuget.org/packages/OwaspHeaders.Core/
65. 21 Best Practices to Improve Security in .NET Core Applications | by raw-hitt | Medium, acceso: marzo 12, 2026, https://medium.com/@rp99452/21-best-practices-to-improve-security-in-net-core-applications-e614251f7280
66. Enforce a Content Security Policy for ASP.NET Core Blazor - Microsoft Learn, acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/aspnet/core/blazor/security/content-security-policy?view=aspnetcore-10.0
67. Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP ..., acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery?view=aspnetcore-8.0
68. In web development projects, should JWT tokens be stored in cookies or localStorage? : r/reactjs - Reddit, acceso: marzo 12, 2026, https://www.reddit.com/r/reactjs/comments/1o66nnf/in_web_development_proje

cts_should_jwt_tokens_be/
69. Hardening ASP.NET Core: HTTP Headers for Security, acceso: marzo 12, 2026, https://asp.net-hacker.rocks/2025/07/23/web-api-headers.html
70. Bypassing client-side controls - PortSwigger, acceso: marzo 12, 2026, https://portswigger.net/burp/documentation/desktop/testing-workflow/vulnerabilities/input-validation/client-side-controls
71. Client-Side Security Threats to be Aware Of | Blog | Digital.ai, acceso: marzo 12, 2026, https://digital.ai/catalyst-blog/client-side-security-threats/
72. Business Logic Vulnerabilities excessive trust in client-side controls | by Harry D - Medium, acceso: marzo 12, 2026, https://medium.com/@d.harish008/business-logic-vulnerabilities-excessive-client-side-controls-c6a1f9a2e6ab
73. JavaScript: client-side vs. server-side validation - Stack Overflow, acceso: marzo 12, 2026, https://stackoverflow.com/questions/162159/javascript-client-side-vs-server-side-validation
74. An Introduction to the Importance of Input Validation in Preventing Security Vulnerabilities, acceso: marzo 12, 2026, https://brightsec.com/blog/an-introduction-to-the-importance-of-input-validation-in-preventing-security-vulnerabilities/
75. how to prevent clients to modify the product price before sending to the server? [closed], acceso: marzo 12, 2026, https://stackoverflow.com/questions/69326268/how-to-prevent-clients-to-modify-the-product-price-before-sending-to-the-server
76. Idempotent requests | Stripe API Reference, acceso: marzo 12, 2026, https://docs.stripe.com/api/idempotent_requests
77. Implementing Idempotency Keys in REST APIs | Zuplo Learning Center, acceso: marzo 12, 2026, https://zuplo.com/learning-center/implementing-idempotency-keys-in-rest-apis-a-complete-guide
78. Implementing Idempotent REST APIs in ASP.NET Core - Milan Jovanović, acceso: marzo 12, 2026, https://www.milanjovanovic.tech/blog/implementing-idempotent-rest-apis-in-aspnetcore
79. Idempotency in REST APIs in ASP.NET Core | by Daniel Alabuja | Medium, acceso: marzo 12, 2026, https://medium.com/@Alabuja/idempotency-in-rest-apis-in-asp-net-core-4a3f30cb8e94
80. How to Implement Idempotency Keys in .NET - OneUptime, acceso: marzo 12, 2026, https://oneuptime.com/blog/post/2026-01-25-implement-idempotency-keys-dotnet/view
81. Let's Build Production-Ready Audit Logs in PostgreSQL | by Sehban Alam | Medium, acceso: marzo 12, 2026, https://medium.com/@sehban.alam/lets-build-production-ready-audit-logs-in-p

ostgresql-7125481713d8
82. PostgreSQL partitioning for event tables in audit logging - AppMaster, acceso: marzo 12, 2026, https://appmaster.io/blog/postgresql-partitioning-event-audit-tables
83. Channels in .NET 8 - DEV Community, acceso: marzo 12, 2026, https://dev.to/kleinhouzin/channels-in-net-8-4hog
84. Job Offloading Pattern with System.Threading.Channels. A way to deal with long-running tasks in .NET - nikiforovall.blog, acceso: marzo 12, 2026, https://nikiforovall.blog/dotnet/async/2024/04/21/job-offloading-pattern.html
85. Channels - .NET | Microsoft Learn, acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/dotnet/core/extensions/channels
86. Package Audit.NET.PostgreSql - GitHub, acceso: marzo 12, 2026, https://github.com/thepirat000/Audit.NET/packages/12977
87. Background tasks with hosted services in ASP.NET Core | Microsoft Learn, acceso: marzo 12, 2026, https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0
88. Managing PostgreSQL partitions with the pg_partman extension - Amazon Aurora, acceso: marzo 12, 2026, https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/PostgreSQL_Partitions.html
89. Managing PostgreSQL partitions with the pg_partman extension - AWS Documentation, acceso: marzo 12, 2026, https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/PostgreSQL_Partitions.html
90. Data archiving and retention in PostgreSQL. Best practices for large datasets, acceso: marzo 12, 2026, https://dataegret.com/2025/05/data-archiving-and-retention-in-postgresql-best-practices-for-large-datasets/
91. .NET 8: What's New for Authentication and Authorization - Auth0, acceso: marzo 12, 2026, https://auth0.com/blog/whats-new-dotnet8-authentication-authorization/